

StockDex — ALM

Conception

Cas d'utilisation · Classes · Séquences

Jeremy Lardet © 2026

1. Diagramme de cas d'utilisation

Acteurs : Joueur, Administrateur, Système (processus de fluctuation). Trois fonctionnalités principales couvertes : Ouverture de booster, Trade, Fluctuation boursière.

Cas d'utilisation principaux :

- Joueur → Ouvrir un booster : inclut [Vérifier solde coins] et [Tirer les cartes pondéré par rareté]
- Joueur → Proposer un trade : inclut [Sélectionner cartes offertes] et [Choisir destinataire]
- Joueur → Accepter / refuser un trade : étend Proposer un trade
- Joueur → Consulter collection : inclut [Afficher cours actuel] et [Afficher sparkline]
- Administrateur → Configurer seuils de marché : étend Système de fluctuation
- Système → Faire fluctuer les cours : job nodecron toutes les 30 s, pousse via Socket.IO

Justification : ce diagramme isole les deux acteurs humains du processus automatisé. La relation <<include>> sur le tirage pondéré est centrale car elle conditionne l'équilibre économique du jeu.

2. Diagramme de classes

Entités métier principales et leurs relations :

Classe	Attributs	Relations
User	id, pseudo, passwordHash, coins, gems, createdAt	1 User → * OwnedCard (collection) User → * Trade (initiator/receiver)
Card	id, name, family (ENUM 7), baseRarity, currentRarity, currentPrice, priceHistory[]	1 Card → * OwnedCard 1 Card → * TradeItem
OwnedCard	id, userId, cardId, acquiredAt	Lien User ↔ Card (agrégation)
Trade	id, initiatorId, receiverId, status (ENUM), createdAt, resolvedAt	1 Trade → * TradeItem (offered / requested)
TradeItem	id, tradeId, cardId, side (offered requested)	Composition de Trade
MarketEvent	id, cardId, oldPrice, newPrice, oldRarity, newRarity, triggeredAt	Log de fluctuation

Justification : la séparation OwnedCard / Card permet à plusieurs joueurs de posséder la même carte sans dupliquer ses données de marché. MarketEvent sert à la fois au sparkline historique et à l'audit du moteur de fluctuation.

3. Diagrammes de séquence

3a — Ouverture d'un booster

Participants : Joueur, React App (client), Express API (POST /boosters/open), MarketEngine, SQLite

- 1. Joueur clique sur «Ouvrir» — React vérifie localement le solde (coins ≥ prix booster)

- 2. POST /boosters/open avec JWT → API vérifie le token et le solde en base
- 3. API interroge MarketEngine.drawCards(n) : tirage pondéré par currentRarity de chaque Card
- 4. API débite les coins, insère n OwnedCard, retourne la liste de cartes
- 5. React déclenche l'overlay BoosterOpenOverlay avec l'animation de révélation
- 6. Collection mise à jour localement, toast de confirmation

Justification : le tirage se fait côté serveur pour garantir l'équité (le client ne peut pas manipuler les probabilités). Le JWT évite toute usurpation.

3b — Proposition et acceptation d'un trade

- 1. Joueur A sélectionne ses cartes offertes + les cartes demandées → POST /trades
- 2. API crée un Trade (status: PENDING) + les TradeItems associés
- 3. Socket.IO émet trade:new sur la room du Joueur B
- 4. Joueur B voit la notification, ouvre le détail du trade
- 5. Joueur B accepte → PATCH /trades/:id {status: accepted}
- 6. API dans une transaction SQLite : transfère les OwnedCard entre les deux joueurs, passe le trade en COMPLETED
- 7. Socket.IO émet trade:resolved sur les deux rooms — les deux clients actualisent leur collection

Justification : la transaction SQLite garantit l'atomicité de l'échange (pas de carte perdue en cas d'erreur réseau). Le broadcast Socket.IO remplace le polling manuel.